

Counting sort

Cos'è?: Il counting sort è un algoritmo di ordinamento iterativo *non comparativo* basato sul conteggio delle occorrenze

Come funziona: Supponiamo di voler ordinare un array A di n elementi con valori compresi tra n e k

1) Creiamo l'array di supporto: Crea un array C lungo $k+1$, inizialmente tutto è zero

2) Conteggio delle occorrenze: Per ogni elemento x in A incrementa $C[x]$

3) Calcolo la prefix sum: Trasforma C in un array cumulativo: ogni $C[i]$ conterrà il numero di elementi $\leq i$

for (int $i=1$; $i \leq k$; $i++$)
 $C[i] = C[i-1] + 1$

4) Costruisco l'array ordinato: Scorro A inversamente e posiziono ogni elemento x nella sua posizione finale usando $C[x]$

Esempio: $A = [4, 2, 2, 8, 3, 3, 1]$

1) Crea C

$A = [4, 2, 2, 8, 3, 3, 1]$

$C = [0, 0, 0, 0, 0, 0, 0, 0, 0]$

2) Itero su ogni A e faccio $C[x]++$

$A = [4, 2, 2, 8, 3, 3, 1]$

$C = [0, 1, 2, 2, 1, 0, 0, 0, 1]$

3) Calcolo la prefix sum

$A = [4, 2, 2, 8, 3, 3, 1]$

$C = [0, 1, 3, 5, 6, 6, 6, 6, 7]$

4) Costruisco l'array ordinato B

$A = [4, 2, 2, 8, 3, 3, 1]$

$C = [0, 1, 3, 5, 6, 6, 6, 6, 7]$

$B = [1, 2, 3, 5, 5, 6, 8]$

i	A[i]	Posizione in B (C[A[i]]-1)	Stato B dopo inserimento	Stato C dopo decremento
6	1	$C[1]-1 = 1-1 = 0$	[1, _ , _ , _ , _ , _ , _ , _ , _]	$C[1] = 1-1 = 0$
5	3	$C[3]-1 = 5-1 = 4$	[1, _ , _ , 3, _ , _ , _ , _ , _]	$C[3] = 5-1 = 4$
4	3	$C[3]-1 = 4-1 = 3$	[1, _ , _ , 3, 3, _ , _ , _ , _]	$C[3] = 4-1 = 3$
3	8	$C[8]-1 = 7-1 = 6$	[1, _ , _ , 3, 3, _ , 8, _ , _]	$C[8] = 7-1 = 6$
2	2	$C[2]-1 = 3-1 = 2$	[1, _ , 2, 3, 3, _ , 8, _ , _]	$C[2] = 3-1 = 2$
1	2	$C[2]-1 = 2-1 = 1$	[1, 2, 2, 3, 3, _ , 8, _ , _]	$C[2] = 2-1 = 1$
0	4	$C[4]-1 = 6-1 = 5$	[1, 2, 2, 3, 3, 4, 8, _ , _]	$C[4] = 6-1 = 5$

È in-place? : No, usiamo memoria in più rispetto all'input

È stabile? : Sì, nell'ultimo passaggio, inserisce gli elementi nell'array di output scorrendo l'array originale da destra a sinistra, mantenendo l'ordine relativo degli elementi uguali. Questo assicura che elementi con lo stesso valore rimangano nell'ordine originale

procedura CountingSort(A, K) CountingSort(A, max(A))

C = vettore di K+1 posizioni 0-based (parte con indice 0)

// inizializzo C (lo riempio di 0)

for i = 0 to K do

 C[i] = 0

// Conto le occorrenze e aggiorno C

for j = 1 to A.length do

 C[A[j]] = C[A[j]] + 1

// Calcolo la prefix-sum all'interno di C, il 1° elemento è già opporto

for i = 1 to K do

 C[i] = C[i] + C[i-1]

B = nuovo vettore di A.length elementi // Array di output

// Aggiorno B scorrendo B all'incontrario e aggiornando C ogni volta

for j = A.length down to 1 do

 B[C[A[j]]] = A[j]

 C[A[j]] = C[A[j]] - 1

return B

Tempi di calcolo: $\Theta(n+k)$ dove :

- n = numero di elementi da ordinare
- k = valore massimo possibile degli elementi

- ✓ **Vantaggi:**
- Tempo lineare - molto veloce specialmente quando K (valore massimo) è piccolo rispetto a n
 - Stabile
 - Facile da implementare
 - Non basato su confronti, quindi può rompere il limite inferiore di $\Theta(n \lg n)$
- X **Svantaggi:**
- Non in-place
 - Se K è molto grande rispetto a n diventa inefficiente
 - Limitato ai numeri interi
 - Non adatto a ordinamenti generici o stringhe complesse